

# Archivos

Profesor: Pablo J Piccoli Bornia

Auxiliares: Daniela Longas - M Victoria Tolosa

# ARCHIVOS



# Archivos

*Un archivo (en inglés, file) es una colección numerada y ordenada de bytes, que almacena datos procesables por un programa.*

- Se almacena en un medio físico (disco rígido, pendrive, etc.).

Un programa en ejecución puede leer y/o escribir datos almacenados en distintos archivos. A su vez, las variables de un programa en ejecución viven en la memoria RAM de la computadora, mucho más rápida que un disco rígido

# Archivos

*En el nombre del archivo suele indicarse, mediante la extensión, la forma en que se organiza el contenido del archivo.*



**Existen dos tipos genéricos de archivos**(según su contenido):

- Archivos binarios: organizan su contenido de distintas maneras (.png, .wav, .mp4, .pdf, .xlsx, .docx)
- Archivos de texto: contienen únicamente caracteres imprimibles (.txt, .py, .html, .xml, .csv, .tex)

Por simplicidad, **vamos a trabajar con archivos de texto**, pero no hay mayores diferencias conceptuales con los archivos binarios.

# Archivos de texto

```
def sumar_divisores ( n): suma
    = 0
    i = 1
    while divisor <= n: if n % i
        == 0:
            suma = suma + i
        i = i + 1
    return suma
```

**suma.py**

Lorem ipsum dolor sit amet,  
consectetur adipiscing elit.  
Fusce interdum pretium ante,  
nec lobortis erat venenatis nec.  
Nullam auctor lectus at enim  
consequat, non tincidunt risus  
ullamcorper. Suspendisse  
potenti.

**lorem.txt**

- Se compone de caracteres codificados en un sistema de codificación (ASCII, UTF-8, etc.), que representan letras, números, símbolos y espacios.
- Los archivos de texto pueden ser creados y editados con cualquier editor de texto (Notepad, Atom, Vim, Sublime, etc.)

.CSV



## CSV

Los **CSV** (Comma Separated Values) son archivos de texto con un formato común para almacenar datos.

```
Nombre,Edad,Profesión,Nacionalidad
Juan,25,Ingeniero,Argentina
María,30,Abogada,Mexicana
Ana,35,Medica,Colombiana
Luis,32,Arquitecto,Peruana
Sofía,27,Periodista,Chilena
Carlos,29,Ingeniero,Brasileño
Laura,31,Economista,Paraguaya
```

- Cada fila representa un registro o una entrada de datos, y las columnas representan los campos o atributos.
- Los valores en un archivo CSV están separados por comas (",") o por otro delimitador (ej: ";").

# CSV - Lectura de Archivos de texto

**open(filename)** → abrimos un archivo en **modo lectura**.

```
f = open ( ' SMSSpamCollection.txt ' )
```

**Nota:** En Windows, puede ser necesario un argumento adicional para reconocer la codificación del texto: `f = open('padron.csv', encoding='utf-8')`

**f.read()** (devuelve un str) → leemos todo el contenido del archivo

```
texto = f.read()      #Cuidado: ¡puede consumir mucha memoria!  
print(texto)
```

**f.readline()** (devuelve un str) → leemos las líneas de a una

```
print (f.readline() )    #imprime ' Nombre , Edad , Profesión , Nacionalidad '  
print(f.readline() )    #imprime ' Juan , 25 , Ingeniero , Argentina '
```

**Forma recomendada** → **iterar** el archivo, leyendo línea a línea:

```
for linea in f :  
    print(linea)
```

# CSV - Escritura de Archivos de texto

`open(filename, 'w')` → abrimos un archivo en **modo escritura** (*w es por write*).

```
f = open('primeros - diez. txt', 'w')
```

`f.write('...')` → instrucción para escribir

```
for i in [1,2,3,4,5,6,7,8,9,10]:  
    f.write(str(i) + '\n')    #\n indica fin de línea
```

se escriben los primeros diez números naturales en el archivo **primeros-diez.txt**, uno por línea.

Al terminar, es buena costumbre **cerrar** el archivo:

```
f.close()
```

# CSV

**#Cuidado:** Los CSV tienen varias limitaciones para representar datos con estructuras complejas

- estructura tabular fija, donde cada fila representa un registro y cada columna representa un campo
- falta de soporte para anidamiento: no admiten la representación de estructuras anidadas o jerárquicas
- no almacenan información sobre los tipos de datos de los campos (todos los valores se tratan como strings)

# JSON



# JSON

**JSON** (JavaScript Object Notation) es un formato estándar para almacenar y representar datos.

```
{
  "carrito1": [
    {
      "marca": "Nike", "modelo": "Air Max
      90", "precio": 60000.00
    },
    {
      "marca": "Adidas", "modelo":
      "Superstar", "precio": 40000.00
    }
  ],
  "carrito2": [
    {
      "marca": "Puma",
      "modelo": "Suede Classic", "precio":
      35000.00
    }
  ]
}
```

- permiten representar estructuras de datos complejas
- almacenan información de los tipos de los datos

# JSON

El formato JSON tienen reglas específicas que determinan los tipos de datos que son **válidos** para las **claves** y para los **valores**.

- Las **claves** deben ser strings(en JSON se escriben entre comillas dobles, ej: *"delivery"*)
- Los **valores** pueden ser String, Número, Array, Booleano, Objeto JSON(ej: *true*)

```
{
  "tamano":"mediana",
  "precio": 1499.99,
  "adicionales":["champinones", "pepperoni", "albahaca"],
  "queso_extra": false,
  "delivery": true,
  "cliente": {
    "nombre": "Paula",
    "telefono": "1133876651",
    "correo": "pau76@em.com"
  }
}
```

# JSON - Lectura

**json.load(file)** → función que carga en un diccionario el contenido del archivo file → **deserialización**

```
import json

# Se abre el archivo JSON
f = open('pedido.json')

pedido = json.load(f)          # devuelve un diccionario con el contenido del objeto JSON
print(pedido['tamano'])        # 'mediana'
print(pedido['precio'])        # 1499.99
print(pedido['adicionales'])    # ['champinones', 'pepperoni', 'albahaca']
print(pedido['queso_extra'])    # False
print(pedido['delivery'])       # True
print(pedido['cliente'])        #
{'nombre': 'Paula', 'telefono': '1133876651', 'mail': 'pau76@em.com'}
```

| JSON                  | Python            |
|-----------------------|-------------------|
| String                | str               |
| Número(entero)        | int               |
| Número(real)          | float             |
| Array                 | list              |
| Booleano (true/false) | bool (True/False) |
| Objeto JSON           | dict              |
| null                  | None              |

**Los tipos de datos de JSON se convierten a tipos de datos en Python**

# JSON - Escritura

**json.dump(dict,file,indent=n)** → función que escribe un objeto JSON en el archivo **file**, a partir de los datos contenidos en **dict**.

→ El parámetro (opcional) **indent** se usa para dar formato al nuevo archivo

```
import json

# Se crean diccionarios con los datos
cliente_json = dict()
cliente_json['nombre'] = 'Manuel'
cliente_json['telefono'] = '1155955953'
cliente_json['mail'] = 'manu14@em.com'

pedido_json = dict()
pedido_json['tamano'] = 'chica'
pedido_json['precio'] = 1299.99
pedido_json['adicionales'] = ['jamon', 'morron']
pedido_json['queso_extra'] = True
pedido_json['delivery'] = False
pedido_json['cliente'] = cliente_json

f = open('nuevo-pedido.json', 'w') # Se abre el archivo destino en modo escritura
json.dump(pedido_json, f, indent=4) # Se escriben los datos como objeto JSON
f.close() # Se cierra el archivo
```

→ Al generar el **archivo**, los tipos de datos de los valores en Python se transforman a tipos de datos JSON (**ej:** True pasa a ser true).

→ Proceso de generar un objeto JSON a partir de un diccionario Python → **serialización**

# JSON - Ejercicio

Considerar la clase **Estudiante** que se describe a continuación:

- `e = Estudiante(nom, leg, prom)`: Crea un nuevo estudiante con el nombre, el número de legajo y el promedio especificados.
- `e.nombre`: nombre del estudiante `e`.
- `e.legajo`: número de legajo del estudiante `e`.
- `e.promedio`: promedio del estudiante `e`.

Implementar la función **serializar\_estudiante(est, nom\_archivo)**, que recibe como parámetros una instancia de la clase **Estudiante** y el nombre de un archivo JSON, y debe escribir en dicho archivo la serialización del objeto `est`.

El siguiente ejemplo ilustra el uso de la función y el contenido que debe generarse:

```
from estudiante import Estudiante
import json

def serializar_estudiante(est, nom_archivo):
    ...

e = Estudiante("Juan Pérez", 10234, 8.75)

serializar_estudiante(e, "estudiante.json")
```

```
{
  "nombre": "Juan Pérez",
  "legajo": 10234,
  "promedio": 8.75
}
```